

GOSPA

AN ENERGY-EFFICIENT HIGH-PERFORMANCE GLOBALLY OPTIMIZED SPARSE CONVOLUTIONAL NEURAL NETWORK ACCELERATOR

Team 19: Sara Ahmad, Mahdi Hassen, Fred Huang, Adil Kazimov, Emon Sarkar



MOTIVATION

Many sparse CNN architectures avoid multiplications of zero activations/weights but introduce architecturally wasteful data transfers that are not necessary in the actual convolution. The GoSPA architecture uses computation reordering to achieve high weight reuse with low hardware and energy cost.

PROBLEM STATEMENT

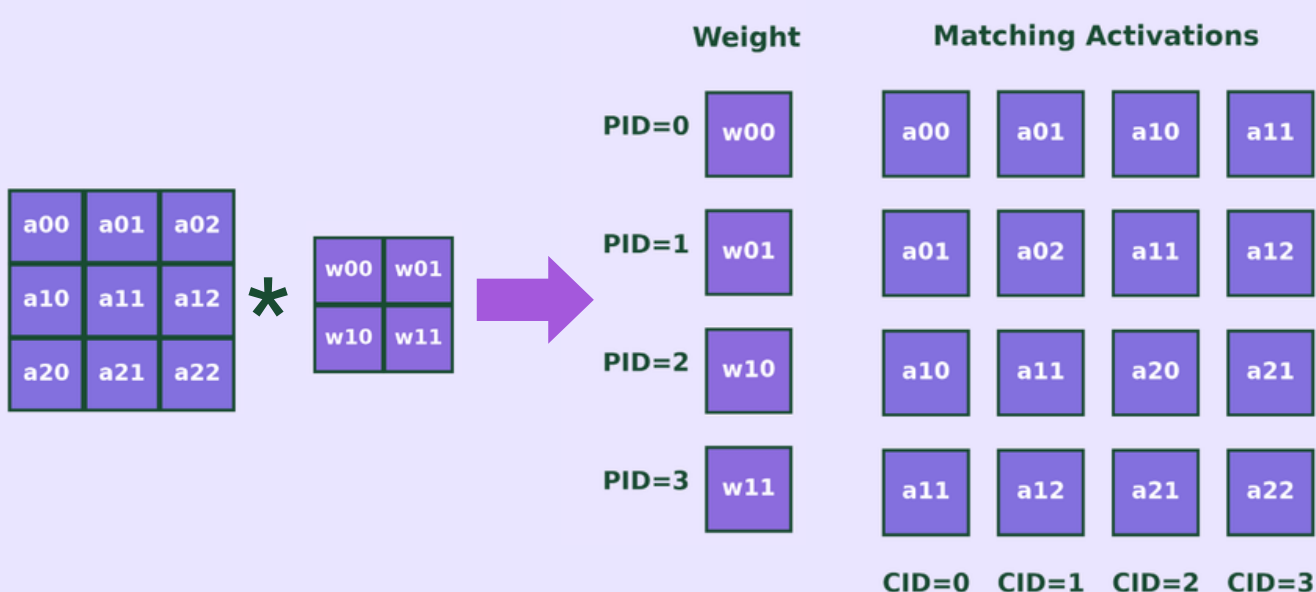
We aim to reproduce the claim that GoSPA handles unstructured sparsity with no extra overhead by exploiting the static nature of weights in a CNN.

C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, "GoSPA: An Energy-Efficient High-Performance Globally Optimized SParse Convolutional Neural Network Accelerator," ISCA, 2021,

PROJECT SCOPE

Implemented a parameterized GoSPA sparse-convolution accelerator containing CSR activation scanning, CID/PID generation, routing module, FIFOs, a multi-lane PE array, pipelined multiplication and CID-indexed accumulation. Python simulators and a compiler provide workload mapping and performance analysis.

COMPUTATION RE-ORDERING

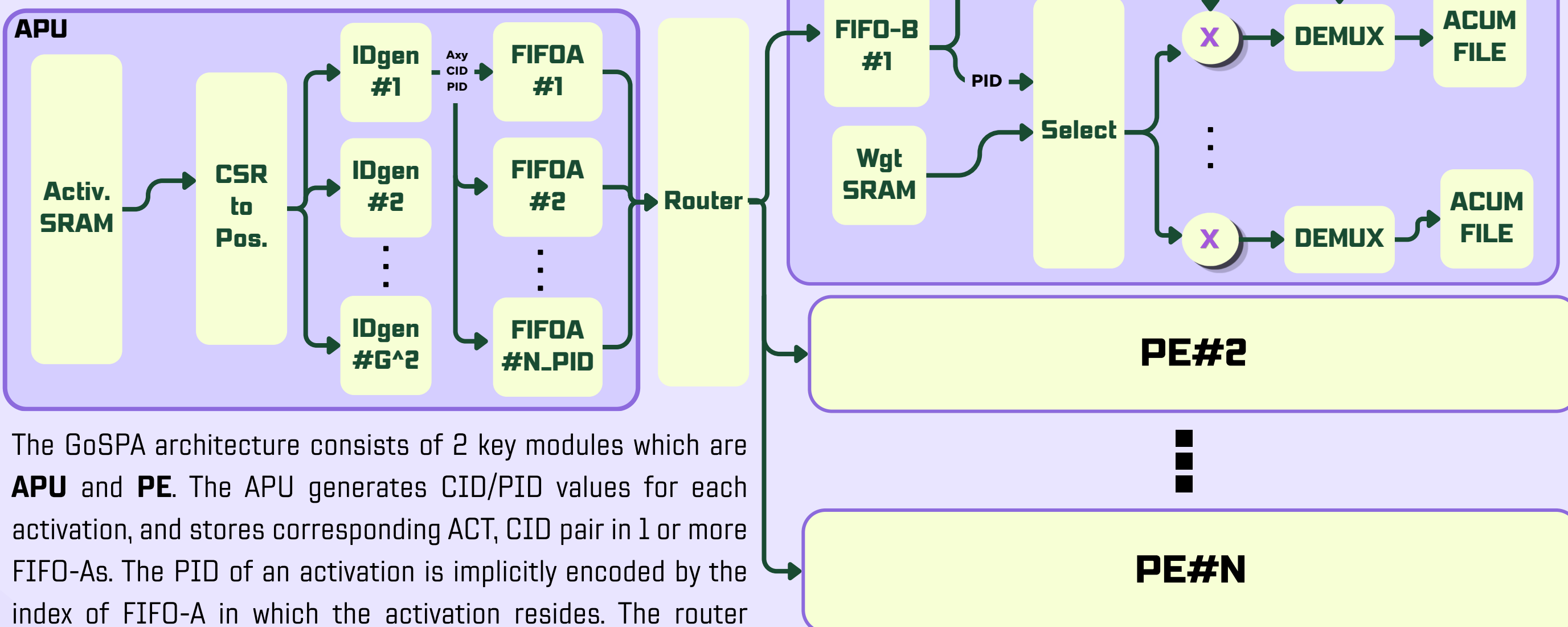


By re-ordering the computation, the unstructured sparsity can be exploited using **on-the-fly intersection** such that only the effectual MACs are performed. Every activation element is tagged by 2 IDs:

- Position ID (**PID**): indices of weights it multiplies with.
- Convolution ID (**CID**): indices of output pixels that this activation lands in.

A weight must be multiplied by all activations with a matching PID. With this in mind, only activations with matching PIDs are streamed into a PE.

TOP LEVEL ARCHITECTURE



The GoSPA architecture consists of 2 key modules which are **APU** and **PE**. The APU generates CID/PID values for each activation, and stores corresponding ACT, CID pair in 1 or more FIFO-As. The PID of an activation is implicitly encoded by the index of FIFO-A in which the activation resides. The router module then routes the elements in the FIFO-As into the corresponding FIFO-Bs. Each FIFO-B corresponds to one PE, which holds one 2D kernel. An element is only routed from FIFO-A to a specific FIFO-B if the corresponding kernel has a weight with a PID that matches the routed activation. Once all the FIFO-As are drained, a new activation can be pushed into the APU.

Once an activation is inside of FIFO-B, the PE fetches the activation and its metadata alongside the weight itself and its PID. Matching activation and weight pairs are multiplied and the results are stored in the accumulator file. If the PID of a newly popped activation is different, the PE fetches a new weight from its SRAM and the process repeats.

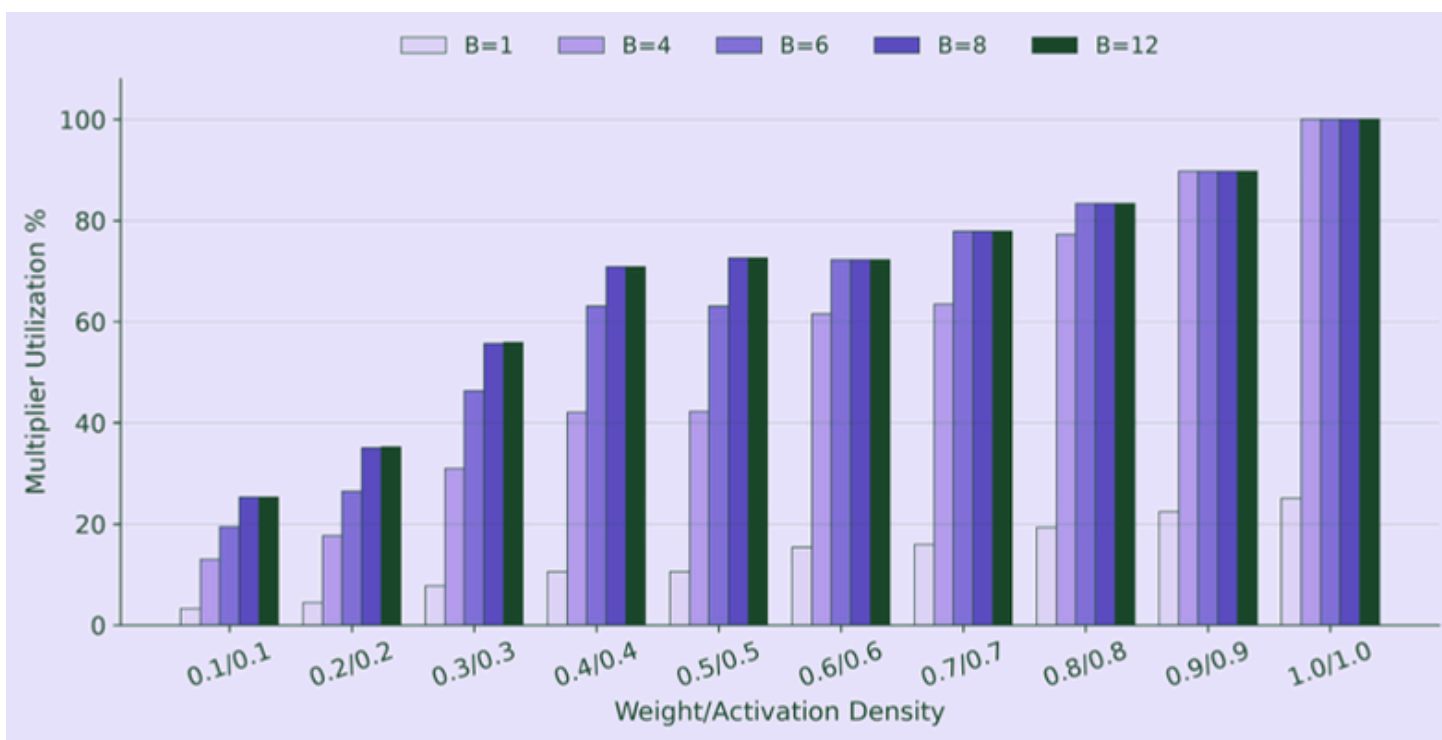
VERIFICATION

- We verified full GoSPA functionality by building a functional python model.
- CocoTB was then used to write unit tests for each module in the design. The modules would use functional model as the source of truth for verification.
- Custom compiler was written to map workloads onto GoSPA
- MobileNetV2 & AlexNet were run on RTL testbench, achieving exact match between RTL outputs and PyTorch golden model.



PERF MODEL : THE ARCH EXPLORER

To enable efficient design space iteration & verification, a python performance model was used to evaluate the performance changes in the hardware, with knobs to control the hardware configurations (e.g., M_LANE, N_PE, BATCH_SIZE). This model simulates the dataflow and gathers various performance metrics. Through the performance model, we revealed that one key factor to boost PE/multiplier utilization is the stage-2 batch size. With the default batch of 1, the PEs will be heavily stalled; with the batch size of M_LANE = 4, the utilization will be capped around weight density.



RESULTS

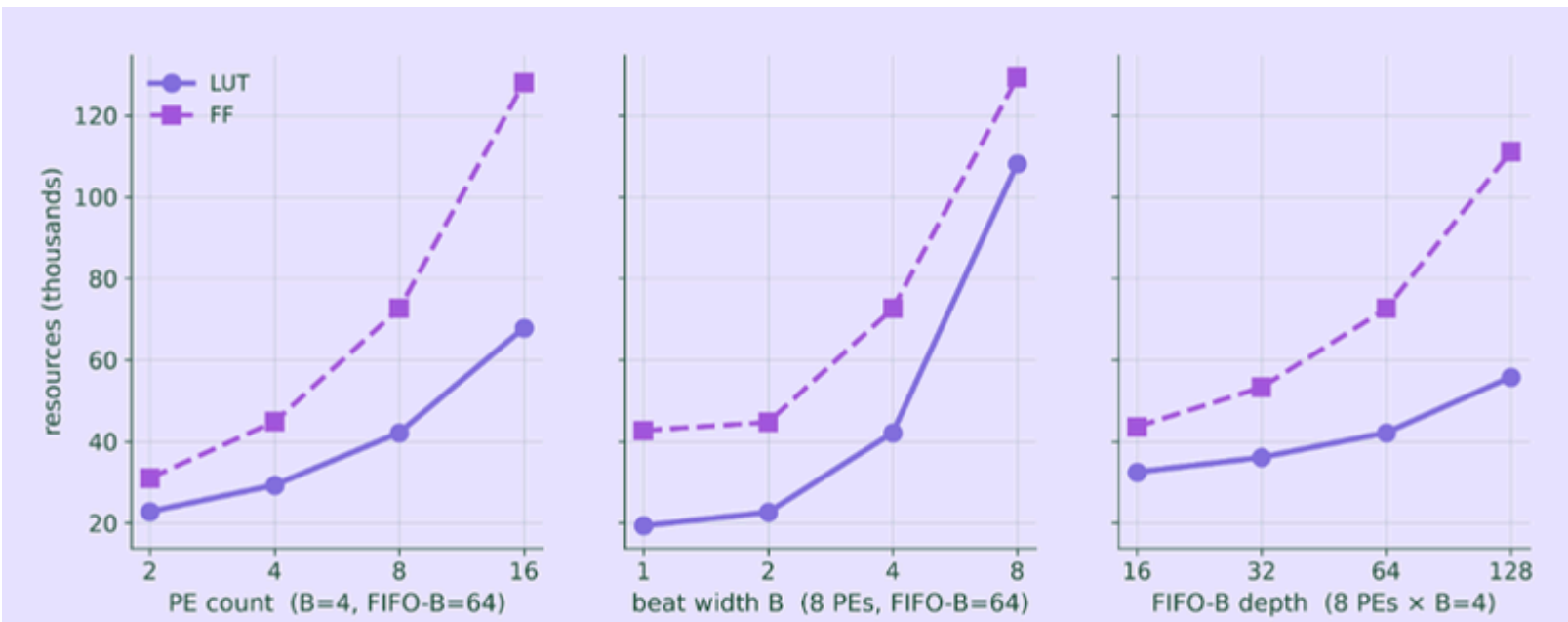
Tools Used:

Software: Python 3.14, PyTorch, NumPy

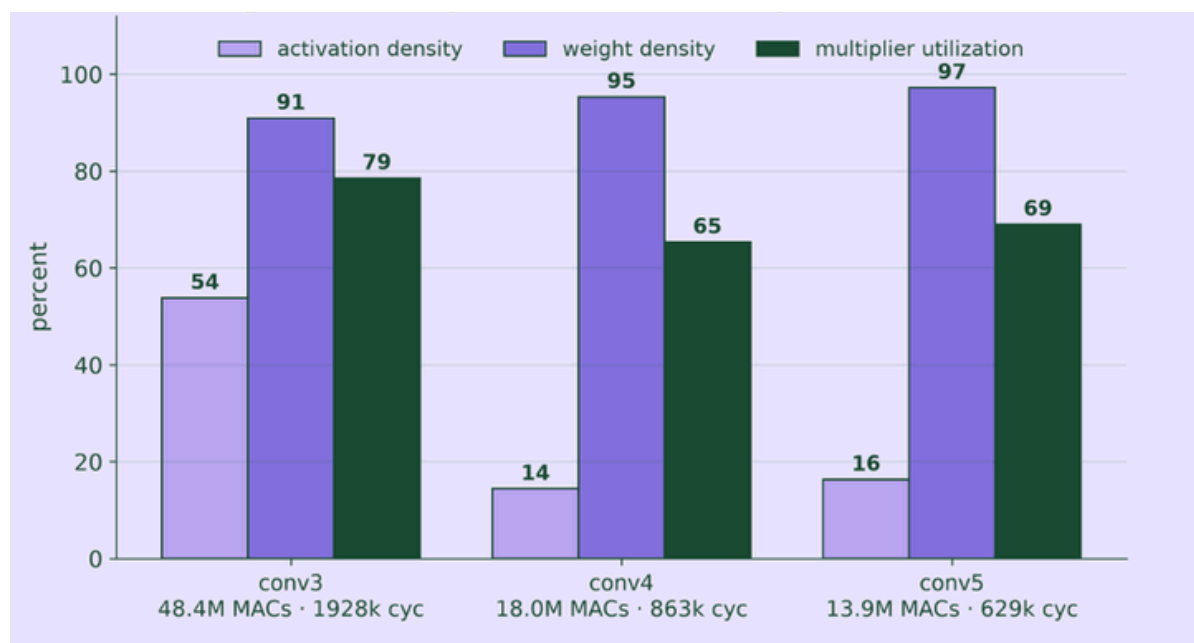
Hardware: SystemVerilog, Verilator, IVerilog, CocoTB, Vivado 2024.2

Synthesis & Implementation on Kria KV260 FPGA

When sweeping across hardware parameters (numPEs, FIFO-B depth, beat width for FIFO-A to B drain) we observe that each parameter affects resource usage as expected:



On the newest version of the RTL, we achieve decent utilization on real-world workloads. As the perf model predicted, lower density causes decreased multiplier utilization, thus affecting performance negatively. Over the coming week, we hope to debug this and achieve consistently high multiplier utilization.



With the current iteration of our architecture, we achieve the following performance stats:

- Fmax: 171Mhz
- Peak ideal OPS (32PE, 4Mul): 43.8 GOPS
- Achieved OPS (MOBILENET): 21.3 GOPs
- Throughput: 256 Frame/s (667,952 cyc)

NEXT STEPS

- Run additional workloads on GoSPA, such as VGG-16 and GoogLeNet
- Run GoSPA through ASIC flow to obtain area and energy metrics.
- Rework scheduling and control logic to maximize throughput of the architecture and replicate paper results
- Optimize critical path to achieve 500Mhz frequency. Currently we are at 171Mhz